

UNIT 5: FUNDAMENTALS OF GRAPH THEORY

Basic Concepts, Applications, Special Properties, Algorithms

Author: Eng. Carlos Andrés Sierra, M.Sc.
cavirguezs@udistrital.edu.co

Full-time Adjunct Professor
Computer Engineering Program
School of Engineering
Universidad Distrital Francisco José de Caldas

2026-II



Course Outline

- 1 Basic Concepts
- 2 Applications of Graphs
- 3 Special Graph Properties
- 4 Algorithms and Analysis



Outline

- 1 Basic Concepts
- 2 Applications of Graphs
- 3 Special Graph Properties
- 4 Algorithms and Analysis



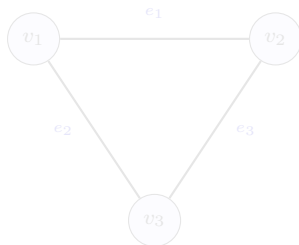
What is a Graph?

Mathematical Definition

A **Graph** $G = (V, E)$ is a discrete mathematical structure used to model pairwise relationships between objects. It consists of a set of **Vertices** (V) and a set of **Edges** (E) connecting them.

Core Components:

- **Vertices (Nodes):**
Represent fundamental entities (e.g., cities, servers, users).
- **Edges (Links):** Represent the relationships or connections between entities (e.g., roads, cables, friendships).



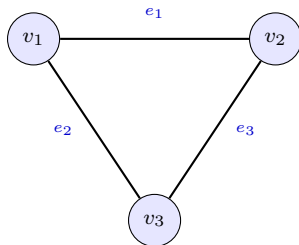
What is a Graph?

Mathematical Definition

A **Graph** $G = (V, E)$ is a discrete mathematical structure used to model pairwise relationships between objects. It consists of a set of **Vertices** (V) and a set of **Edges** (E) connecting them.

Core Components:

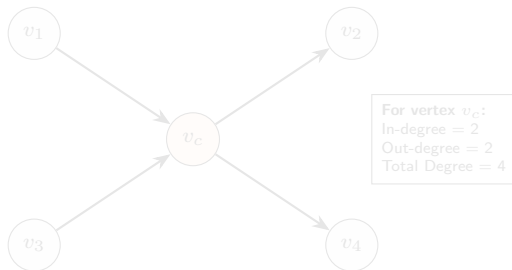
- **Vertices (Nodes):**
Represent fundamental entities (e.g., cities, servers, users).
- **Edges (Links):** Represent the relationships or connections between entities (e.g., roads, cables, friendships).



Incidence and Degree

Vertex Degree ($\deg(v)$)

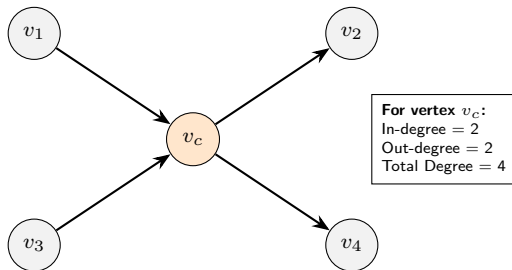
The **degree of a vertex** is the total number of edges incident to (connecting to) that vertex. In directed graphs, it is divided into *in-degree* and *out-degree*.



Incidence and Degree

Vertex Degree ($\deg(v)$)

The **degree of a vertex** is the total number of edges incident to (connecting to) that vertex. In directed graphs, it is divided into *in-degree* and *out-degree*.



Walks, Trails, Paths, and Circuits

Graph Traversals

Moving through a graph sequentially creates different topological sequences based on specific repetition constraints.

- **Walk:** Any sequence of alternating vertices and edges. (No restrictions).
- **Trail:** A walk in which all edges are distinct. (Vertices can be repeated).
- **Path:** A trail in which all vertices (and thus edges) are distinct.

Cycles and Circuits

- **Circuit:** A *trail* that starts and ends at the exact same vertex.
- **Cycle:** A *path* that starts and ends at the exact same vertex (no other repeated vertices).

Walks, Trails, Paths, and Circuits

Graph Traversals

Moving through a graph sequentially creates different topological sequences based on specific repetition constraints.

- **Walk:** Any sequence of alternating vertices and edges. (No restrictions).
- **Trail:** A walk in which all edges are distinct. (Vertices can be repeated).
- **Path:** A trail in which all vertices (and thus edges) are distinct.

Cycles and Circuits

- **Circuit:** A *trail* that starts and ends at the exact same vertex.
- **Cycle:** A *path* that starts and ends at the exact same vertex (no other repeated vertices).

Walks, Trails, Paths, and Circuits

Graph Traversals

Moving through a graph sequentially creates different topological sequences based on specific repetition constraints.

- **Walk:** Any sequence of alternating vertices and edges. (No restrictions).
- **Trail:** A walk in which all edges are distinct. (Vertices can be repeated).
- **Path:** A trail in which all vertices (and thus edges) are distinct.

Cycles and Circuits

- **Circuit:** A *trail* that starts and ends at the exact same vertex.
- **Cycle:** A *path* that starts and ends at the exact same vertex (no other repeated vertices).

Subgraphs and Graph Isomorphism

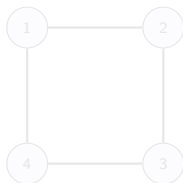
Subgraphs

A graph H is a subgraph of G if its vertices and edges form subsets of those in G .

$$V(H) \subseteq V(G) \text{ and } E(H) \subseteq E(G)$$

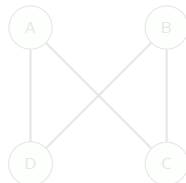
Isomorphism

Two graphs are **isomorphic** if they contain the exact same topological structure, despite drawing variations or different vertex labels.



Graph G_1

$\cong$



Graph G_2

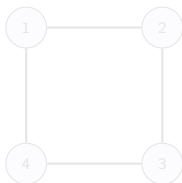


Subgraphs and Graph Isomorphism

Subgraphs

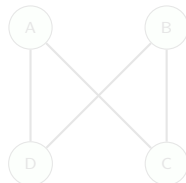
A graph H is a subgraph of G if its vertices and edges form subsets of those in G .

$$V(H) \subseteq V(G) \text{ and } E(H) \subseteq E(G)$$



Graph G_1

$\cong$



Graph G_2

Isomorphism

Two graphs are **isomorphic** if they contain the exact same topological structure, despite drawing variations or different vertex labels.

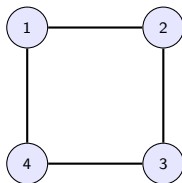


Subgraphs and Graph Isomorphism

Subgraphs

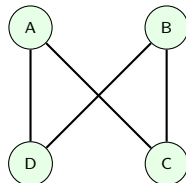
A graph H is a subgraph of G if its vertices and edges form subsets of those in G .

$$V(H) \subseteq V(G) \text{ and } E(H) \subseteq E(G)$$



Graph G_1

$\cong$



Graph G_2

Isomorphism

Two graphs are **isomorphic** if they contain the exact same topological structure, despite drawing variations or different vertex labels.



Outline

- 1 Basic Concepts
- 2 Applications of Graphs**
- 3 Special Graph Properties
- 4 Algorithms and Analysis



Modeling Real-World Problems

The Power of Abstraction

Graphs reduce complex systems into pure structural logic, enabling the application of universal algorithms to solve localized domain problems.

Common Network Topologies:

- **Transportation Networks:** Vertices = Intersections/Airports, Edges = Roads/Flights (Weights = Distance/Cost).
- **Computer Networks:** Vertices = Routers/Switches, Edges = Fiber Links (Weights = Latency/Bandwidth).
- **Social Networks:** Vertices = Users, Edges = Friendships/Followers (Directed vs. Undirected).
- **Biological Systems:** Protein-protein interactions, neural mappings.



Modeling Real-World Problems

The Power of Abstraction

Graphs reduce complex systems into pure structural logic, enabling the application of universal algorithms to solve localized domain problems.

Common Network Topologies:

- **Transportation Networks:** Vertices = Intersections/Airports, Edges = Roads/Flights (Weights = Distance/Cost).
- **Computer Networks:** Vertices = Routers/Switches, Edges = Fiber Links (Weights = Latency/Bandwidth).
- **Social Networks:** Vertices = Users, Edges = Friendships/Followers (Directed vs. Undirected).
- **Biological Systems:** Protein-protein interactions, neural mappings.



Outline

- 1 Basic Concepts
- 2 Applications of Graphs
- 3 Special Graph Properties**
- 4 Algorithms and Analysis



Eulerian and Hamiltonian Constraints

Eulerian Paths / Circuits

A traversal that visits **every edge** in the graph exactly once.

- **Euler Circuit:** Exists iff every vertex has an *even degree*.
- **Euler Path:** Exists iff exactly zero or two vertices have an *odd degree*.

Hamiltonian Paths / Circuits

A traversal that visits **every vertex** in the graph exactly once.

- **Constraint:** Finding a Hamiltonian circuit is an **NP-Complete** problem (e.g., The Traveling Salesperson Problem).



Eulerian and Hamiltonian Constraints

Eulerian Paths / Circuits

A traversal that visits **every edge** in the graph exactly once.

- **Euler Circuit:** Exists iff every vertex has an *even degree*.
- **Euler Path:** Exists iff exactly zero or two vertices have an *odd degree*.

Hamiltonian Paths / Circuits

A traversal that visits **every vertex** in the graph exactly once.

- **Constraint:** Finding a Hamiltonian circuit is an **NP-Complete** problem (e.g., The Traveling Salesperson Problem).



Outline

- 1 Basic Concepts
- 2 Applications of Graphs
- 3 Special Graph Properties
- 4 Algorithms and Analysis



Basic Graph Algorithms & Complexity Analysis

Graph Traversals and Pathfinding

Algorithm performance in graphs is dictated by the combined total of Vertices $|V|$ and Edges $|E|$.

Fundamental Graph Algorithms Cost Matrices

Algorithm	Primary Use Case	Time Complexity	Type
BFS (Breadth-First)	Shortest unweighted path	$O(V + E)$	Traversal
DFS (Depth-First)	Cycle detection, Maze	$O(V + E)$	Traversal
Dijkstra's	Shortest weighted path	$O(E \log V)$	Greedy
Kruskal's / Prim's	Minimum Spanning Tree	$O(E \log V)$	Greedy



Basic Graph Algorithms & Complexity Analysis

Graph Traversals and Pathfinding

Algorithm performance in graphs is dictated by the combined total of Vertices $|V|$ and Edges $|E|$.

Fundamental Graph Algorithms Cost Matrices

Algorithm	Primary Use Case	Time Complexity	Type
BFS (Breadth-First)	Shortest unweighted path	$O(V + E)$	Traversal
DFS (Depth-First)	Cycle detection, Maze	$O(V + E)$	Traversal
Dijkstra's	Shortest weighted path	$O(E \log V)$	Greedy
Kruskal's / Prim's	Minimum Spanning Tree	$O(E \log V)$	Greedy



Basic Graph Algorithms & Complexity Analysis

Memory Representation Overhead

- **Adjacency Matrix:** $O(|V|^2)$ space (Best for dense graphs).
- **Adjacency List:** $O(|V| + |E|)$ space (Best for sparse graphs).



Outline

- 1 Basic Concepts
- 2 Applications of Graphs
- 3 Special Graph Properties
- 4 Algorithms and Analysis



Thanks!

Questions?



Repo: <https://github.com/EngAndres/ud-public/tree/main/courses/computer-sciences-ii>

